

Progress Project SDA
Pengembangan Aplikasi Web Interaktif untuk Visualisasi Graf Molekul
dan Eksplorasi Properti Kuantum pada Dataset QM9



DOSEN PENGAMPU:

Yuni Rosita Dewi, M.Si.

Dr. Heri Purnawan, S.Si., M.Si.

DISUSUN OLEH:

M. Habiburrohman Al-Fathir (24031554145)

Arzha Dwi Ananda Anhar Wijaya (25031554264)

Muhammad Pasha Paksi Pasukopati (25031554019)

PROGRAM STUDI S1 SAINS DATA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA

2026

Daftar Isi

1	PENDAHULUAN	2
2	TUJUAN PROYEK	3
3	STRUKTUR DATA YANG DIGUNAKAN	4
3.1	Graf dengan <i>Adjacency List</i>	4
3.2	<i>Hash Map</i> Indeks	4
3.3	<i>Hash Map</i> Pendukung	5
3.4	<i>Stack</i> pada <i>Parser SMILES</i>	5
3.5	Cache <i>Pickle (Lazy Load)</i>	5
4	OUTPUT YANG DIHARAPKAN	7
4.1	Antarmuka Utama	7
4.2	Visualisasi Graf Molekul	7
4.3	Pencarian Berlapis	9
4.4	<i>Fallback</i> Render dari Notasi	11
4.5	Mode Bandingkan	11
4.6	Performa yang Diharapkan	12
5	ALGORITMA / URUTAN Pengerjaan	13
5.1	Algoritma 1: Membangun Indeks (<i>index_builder.py</i>)	13
5.2	Algoritma 2: Pencarian Molekul (<i>search</i>)	13
5.3	Algoritma 3: Buka Detail Molekul (<i>Lazy Load</i>)	14
5.4	Algoritma 4: Inferensi Ikatan (<i>infer_bonds</i>)	15
5.5	Algoritma 5: Tata Letak 2D (<i>compute_2d_layout</i>)	15
5.6	Algoritma 6: Render <i>Fallback</i> dari Notasi	16
6	LAMPIRAN	17
6.1	Repositori Kode	17
6.2	Cara Menjalankan	17

BAB 1

PENDAHULUAN

Dalam ilmu komputer, graf merupakan struktur data fundamental untuk memodelkan keterhubungan antar elemen. Setiap elemen direpresentasikan sebagai titik (*node*), sedangkan garis penghubung antar titik disebut *edge*. Salah satu ranah yang menggunakan representasi ini secara alami adalah kimia komputasional. Setiap molekul dapat dipandang sebagai graf, dengan atom berperan sebagai *node* dan ikatan kimia sebagai *edge*.

Latar belakang proyek ini bermula dari paper di jurnal *npj Computational Materials* tahun 2026 dengan judul “Adaptive edge-aware graph convolutional with multi-task learning for simultaneous prediction of material properties” [1]. Pada paper tersebut, dataset QM9 dipakai sebagai *benchmark* dan setiap molekulnya dimodelkan sebagai graf untuk diolah menggunakan *graph neural network* (GNN). Hal yang menjadi sorotan kami adalah kebutuhan dasar yang sering luput dari perhatian. Sebelum dataset QM9 dapat diumpankan ke proses *learning*, data tersebut perlu dikelola terlebih dulu, mulai dari pembacaan, pengindeksan, sampai visualisasi. Di tahap inilah peran Struktur Data dan Algoritma menjadi penting.

QM9 mencakup 133.885 molekul organik kecil, masing-masing dilengkapi dengan 16 properti kuantum yang dihitung melalui metode DFT. File mentahnya berformat `.xyz` dan hanya memuat koordinat 3D atom **tanpa menyertakan informasi ikatan**. Akibatnya, pengelolaan dataset ini memerlukan empat hal pokok. Pertama, kemampuan mencari molekul dengan cepat dari ratusan ribu file. Kedua, inferensi ikatan dari koordinat 3D. Ketiga, penataan graf 2D agar mudah dibaca. Terakhir, strategi penyimpanan yang mencegah perhitungan ulang.

Aplikasi web monolitik dikembangkan dalam proyek ini dengan menerapkan beberapa konsep SDA, yaitu struktur graf, *hash map*, algoritma layout berbasis gaya (Kamada-Kawai dan Fruchterman-Reingold), serta strategi *caching* berlapis. Tujuan akhirnya adalah menampilkan molekul QM9 sebagai graf 2D yang interaktif. Penilaian proyek tidak diarahkan ke kualitas model prediksi, melainkan ke seberapa baik pemilihan struktur data dan algoritma membuat eksplorasi 134 ribu molekul tetap responsif. Targetnya adalah waktu *lookup* yang berada di kisaran sub-detik, dan waktu *render* satu molekul dalam beberapa puluh milidetik.

BAB 2

TUJUAN PROYEK

Permasalahan utama yang diangkat proyek ini berkutat di seputar **searching dan optimasi akses** terhadap dataset besar yang berisi 134 ribu file `.xyz`. Pengguna harus mampu mencari molekul lewat rumus kimia maupun notasi SMILES, kemudian melihat tampilan graf 2D-nya beserta keenam belas properti kuantum. Semua proses tersebut perlu berlangsung dalam waktu yang masih nyaman untuk eksplorasi interaktif, dengan target di bawah 200 ms per query.

Berikut tujuan-tujuan spesifik yang ingin dicapai:

1. **Pencarian molekul yang konstan terhadap ukuran dataset.** Operasi lookup berbasis ID, rumus kimia, ataupun SMILES wajib dieksekusi dalam $O(1)$, terlepas apakah dataset memuat 2.000 atau 134.000 molekul. Pendekatan ini membebaskan sistem dari kewajiban memindai folder mentah pada setiap query.
2. **Optimasi waktu *startup*.** Memproses ulang 134 ribu file `.xyz` pada tiap kali aplikasi dinyalakan jelas tidak realistis. Solusinya, sistem hanya membangun *hash map* indeks satu kali pada tahap setup, lalu memuat indeks tersebut langsung dari *disk* pada peluncuran-peluncuran berikutnya.
3. **Pembentukan graf dari data spasial.** Koordinat 3D atom dikonversi menjadi struktur graf $G = (V, E)$ melalui algoritma yang memanfaatkan jarak kovalen. Setelah itu, posisi tiap atom pada bidang 2D dihitung agar tata letaknya stabil saat divisualisasikan.
4. ***Fallback* pembentukan graf.** Beberapa skenario bisa terjadi, misalnya pengguna mengetikkan H2O9 yang tidak ada di alam, atau ingin melihat hasil interpretasi sembarang notasi SMILES. Pada situasi semacam ini, sistem masih sanggup menggambar graf naif, hanya saja tanpa informasi koordinat dan properti kuantum.

BAB 3

STRUKTUR DATA YANG DIGUNAKAN

3.1 Graf dengan *Adjacency List*

Tiap molekul digambarkan sebagai graf $G = (V, E)$. Himpunan V memuat seluruh atom yang berperan sebagai *node*, sementara E berisi ikatan kimia yang berfungsi sebagai *edge*. Pembuatan struktur ini diserahkan kepada NetworkX (`nx.Graph`), yang secara internal sudah memakai *adjacency list*.

Pertimbangan memilih *adjacency list*. Karakter dasar graf molekul adalah **sparse**. Tiap atom rata-rata hanya berhubungan dengan dua atom lain, sehingga *degree*-nya berkisar di angka 2 saja. Jumlah ini jauh dari batas atas $n - 1$. Pada molekul QM9 yang jumlah atomnya tidak melebihi 29 (maksimal 9 atom berat plus hidrogen), sebuah *adjacency matrix* 29×29 akan banyak terisi nol dan boros memori. Sebaliknya, *adjacency list* hanya mencatat tetangga yang benar-benar tersambung. Ruang yang dibutuhkan pun mengecil, dari $O(|V|^2)$ menjadi $O(|V| + |E|)$.

Ilustrasi. Molekul air H_2O memiliki tiga atom (O, H, H) dan dua ikatan (O–H, O–H). Representasi *adjacency list*-nya adalah:

```
O : [H1, H2]
H1: [O]
H2: [O]
```

Tiap entri hanya menyimpan tetangga yang sungguh-sungguh tersambung, sehingga total ruang yang dipakai cuma sebanyak $2|E|$ slot. Bandingkan dengan *adjacency matrix* 3×3 yang akan memuat 9 sel, dan 5 di antaranya bernilai nol.

3.2 *Hash Map* Indeks

Optimasi searching ditopang oleh tiga *hash map* (atau `dict` dalam Python) yang dibangun pada tahap setup, kemudian diserialisasikan ke berkas `index.json`. Ketiganya sebagai berikut:

- `meta: mol_id` $\rightarrow$ metadata yang mencakup formula, SMILES, nama, dan 16 properti kuantum. Berkat struktur ini, lookup berbasis ID molekul berjalan dalam $O(1)$.
- `formula_idx`: rumus kimia dalam notasi Hill (contoh: C_6H_6) $\rightarrow$ daftar `mol_id`. Saat pengguna mengetik H_2O , hasilnya seluruh molekul dengan rumus tersebut bisa langsung diperoleh tanpa perlu memindai dataset.

- `smiles_idx`: SMILES $\rightarrow$ `mol_id`. Berperan untuk pencarian berdasarkan notasi struktural.

Pertimbangan memilih *hash map*. Kebutuhan utama yang harus dipenuhi adalah *lookup* cepat dengan kunci tertentu (ID, formula, atau SMILES). *Hash map* cocok untuk skenario ini karena rata-rata kompleksitasnya $O(1)$. Bandingkan dengan pencarian linier yang berkompleksitas $O(n)$ atau *binary search tree* yang $O(\log n)$. Untuk dataset 134 ribu entri, kedua alternatif tersebut jelas terlalu lambat.

3.3 *Hash Map* Pendukung

Selain ketiga *hash map* indeks utama, sistem juga memanfaatkan beberapa *hash map* berukuran kecil untuk operasi-operasi spesifik:

- `COVALENT_RADII`: elemen kimia $\rightarrow$ jari-jari kovalen dalam satuan Angstrom. *Hash map* ini diakses oleh algoritma inferensi ikatan dengan kompleksitas $O(1)$.
- `SMILES_TO_NAME`: SMILES populer $\rightarrow$ nama yang umum dikenal, sebagai contoh “CCO” diterjemahkan menjadi Etanol. Resolusi nama secara lokal didahulukan sebelum sistem menghubungi API eksternal.
- `names_cache.json`: hasil resolusi nama dari layanan PubChem, disimpan secara permanen di *disk* agar tidak perlu meminta ulang.
- `ATOM_COLORS` (di sisi *frontend*): elemen $\rightarrow$ kode warna heksadesimal untuk pewarnaan tiap *node*.

3.4 *Stack* pada Parser SMILES

Saat sistem perlu menggambar molekul yang tidak ada dalam dataset (jalur *fallback* pada Algoritma 6), notasi SMILES seperti `CC(C)O` harus dibedah satu per satu. Tantangannya muncul pada bagian *branch* yang ditandai oleh tanda kurung. Setiap kali parser menjumpai `(`, atom acuan saat itu perlu disimpan agar nantinya bisa kembali ke titik tersebut ketika `)` ditemukan. Pola simpan dan ambil ulang dengan urutan terbalik ini cocok dijawab oleh struktur *stack*.

Pertimbangan memilih *stack*. Sifat LIFO (*last in, first out*) milik *stack* sejalan dengan aturan *branch* pada SMILES yang bersarang. Operasi `push` dan `pop` sama-sama berkompleksitas $O(1)$. Kalau dipaksa memakai *queue* (FIFO), urutan kembalinya jadi salah dan struktur *branch* tidak bisa direkonstruksi.

3.5 Cache Pickle (*Lazy Load*)

Cache yang disimpan di `cache.pkl` berstruktur *hash map* dengan `mol_id` sebagai kunci dan molekul lengkap (graf, tata letak 2D, beserta properti) sebagai nilainya. Pengisian cache

berlangsung secara bertahap. Hanya molekul yang sempat dibuka oleh pengguna yang ditambahkan ke dalamnya. Saat pertama kali dipanggil, file `.xyz` terkait di-*parse* dari *disk*. Akses berikutnya untuk molekul yang serupa cukup mengambil dari memori, dengan kompleksitas $O(1)$.

Pertimbangan memilih pickle. Pickle merupakan format biner bawaan Python yang proses *serialize* dan *deserialize*-nya cepat. Format ini juga akomodatif terhadap struktur data kompleks seperti objek bersarang dan *numpy array*. Karena tidak menuntut definisi *schema* secara eksplisit, format ini tergolong ringan untuk dipakai pada skala data sebesar ini.

Pertimbangan memilih lazy load. Pendekatan alternatifnya adalah *eager precompute*, yaitu menghitung graf dan tata letak untuk seluruh 134 ribu molekul saat startup. Cara tersebut menjanjikan akses cepat untuk semua molekul, akan tetapi tiap kali aplikasi dinyalakan kembali, sistem harus menunggu beberapa menit dan menyimpan cache yang ukurannya bisa mencapai gigabyte. Padahal, dalam pemakaian nyata pengguna umumnya hanya membuka belasan molekul per sesi. Lewat *lazy load*, biaya komputasi dibayar hanya ketika dibutuhkan. Konsekuensinya, hit pertama untuk satu molekul memang lebih lambat (40–60 ms), namun hit-hit berikutnya berjalan dalam orde milidetik berkat cache.

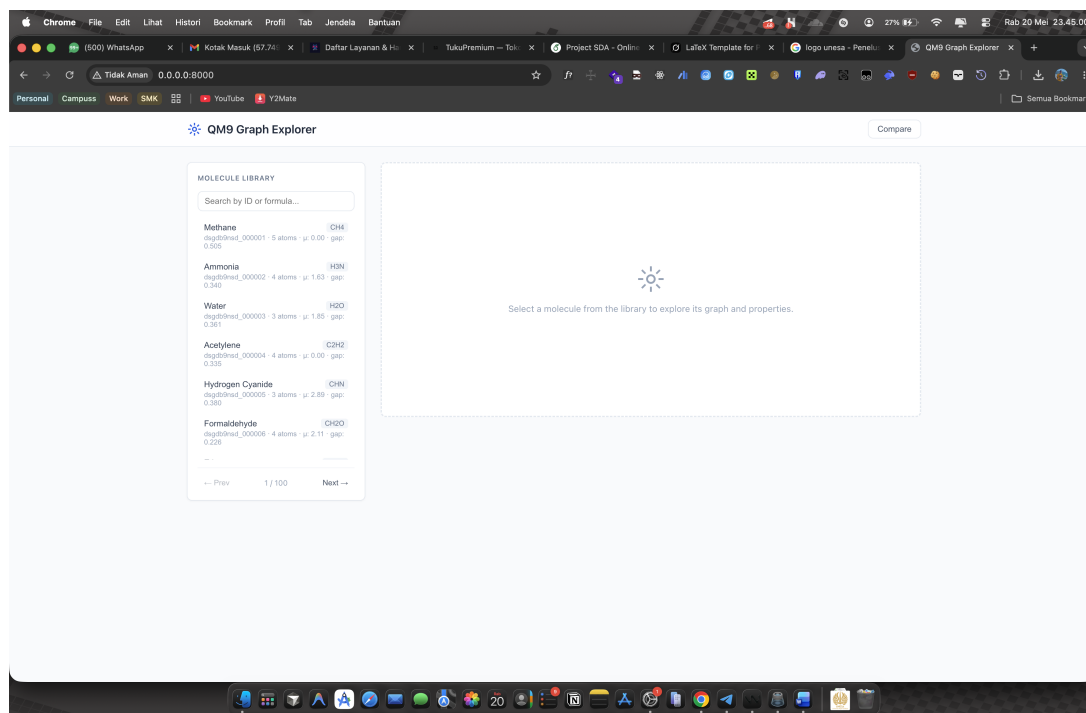
BAB 4

OUTPUT YANG DIHARAPKAN

Hasil akhir sistem berwujud aplikasi web yang dapat diakses melalui alamat `http://localhost:8000`. Tata letaknya dibagi dua. Sebelah kiri ditempati daftar molekul yang berfungsi sebagai sidebar, sedangkan panel sebelah kanan dipakai untuk menampilkan visualisasi graf 2D.

4.1 Antarmuka Utama

Pada halaman utama, pengguna disuguhkan daftar 133.885 molekul yang dibagi ke dalam paginasi sebesar 20 entri per halaman. Di bagian atas tersedia kolom pencarian dengan respon *debounced* 250 ms untuk menghindari beban berlebih. Tombol “Render dari Notasi” disediakan sebagai jalur *fallback*, sementara tombol “Bandingkan” di pojok kanan atas dipakai pengguna untuk menyandingkan hingga empat molekul sekaligus.

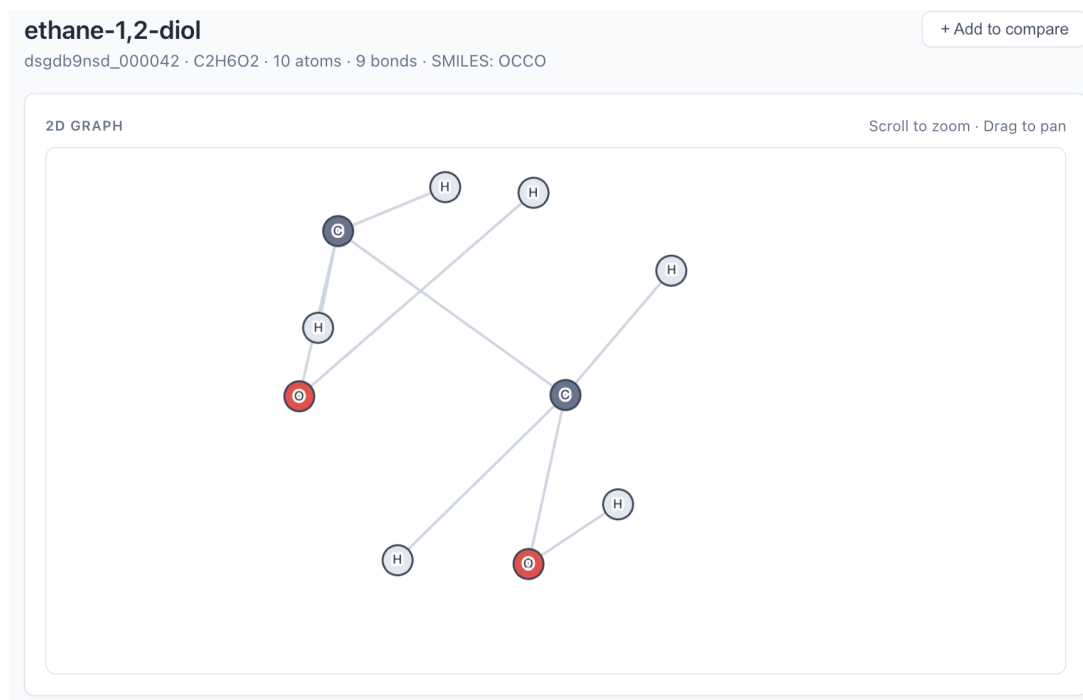


Gambar 4.1: Tampilan utama: daftar molekul (kiri) dan area visualisasi (kanan).

4.2 Visualisasi Graf Molekul

Begitu pengguna memilih satu molekul tertentu, sistem segera menampilkan graf 2D-nya melalui *library* Cytoscape.js. Tiap atom (*node*) diberi warna sesuai elemen kimianya, dengan ketentuan C berwarna abu-abu, N biru, O merah, F hijau, serta H putih. Adapun setiap ikatan

(*edge*) digambarkan sebagai garis penghubung. Pengguna dapat melakukan zoom melalui *scroll* dan menggeser tampilan dengan *drag*.



Gambar 4.2: Graf 2D molekul beserta panel properti kuantum.

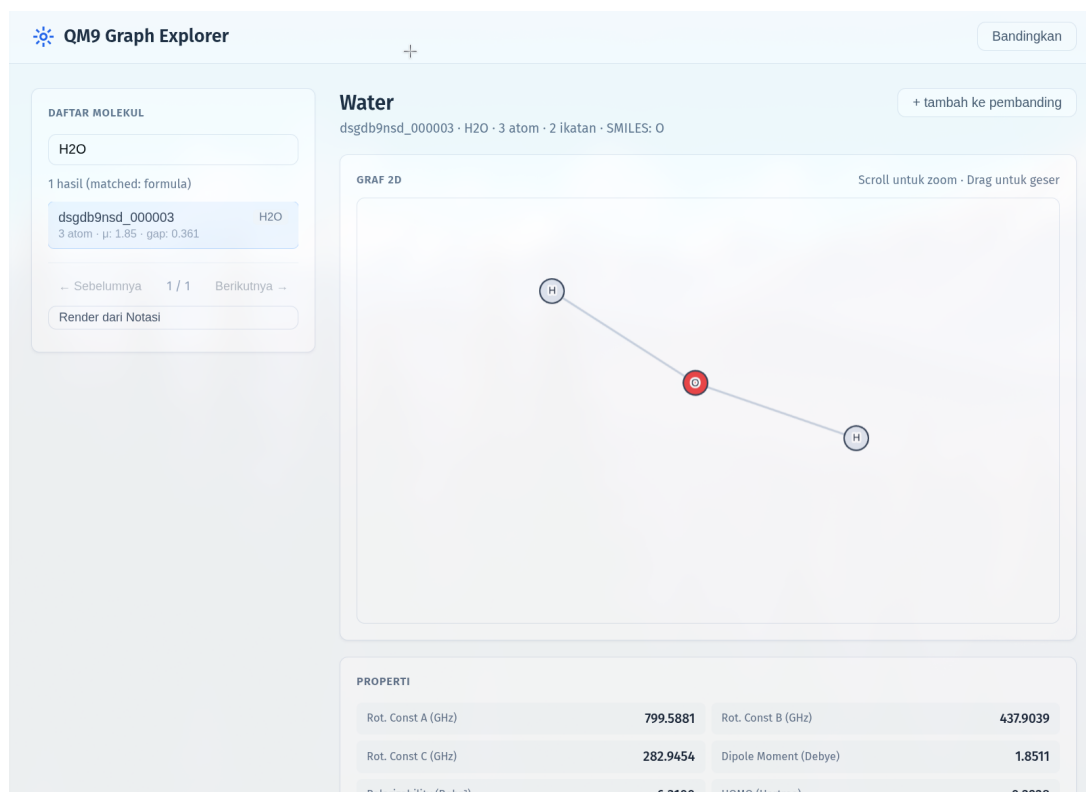
Pada panel di bawah graf, sistem menampilkan ke-16 properti kuantum yang dihasilkan dari perhitungan DFT, mencakup energi orbital HOMO/LUMO, energi gap, momen dipol, polarisabilitas, hingga energi internal pada 0 K. Nilai-nilai ini diambil langsung dari hash map *meta* sehingga akses untuk satu molekul berlangsung dalam $O(1)$.

PROPERTIES			
Rot. Const A (GHz)	799.5881	Rot. Const B (GHz)	437.9039
Rot. Const C (GHz)	282.9454	Dipole Moment (Debye)	1.8511
Polarizability (Bohr ³)	6.3100	HOMO (Hartree)	-0.2928
LUMO (Hartree)	0.0687	HOMO-LUMO Gap (Hartree)	0.3615
Spatial Extent (Bohr ²)	19.0002	ZPVE (Hartree)	0.0214
U ₀ (Hartree)	-76.4047	U (Hartree)	-76.4019
H (Hartree)	-76.4009	G (Hartree)	-76.4223
Cv (cal/mol-K)	6.0020		

Gambar 4.3: Detail panel 16 properti kuantum dan daftar atom penyusun molekul.

4.3 Pencarian Berlapis

Kolom pencarian mengakomodasi tiga jenis input yang sering dipakai: rumus kimia, notasi SMILES, dan substring nama atau `mol_id`. Untuk rumus kimia seperti H2O, sistem mengembalikan seluruh molekul dengan formula tersebut. Pada notasi SMILES seperti CCO, hasil yang muncul adalah satu molekul yang persis cocok (etanol). Sedangkan substring dipakai sebagai jalur *fallback* ketika dua tier sebelumnya *miss*.



Gambar 4.4: Pencarian Tier 1: input formula H2O mengembalikan daftar molekul lewat hash map `formula_idx`.

QM9 Graph Explorer Bandingkan

DAFTAR MOLEKUL

CCO

1 hasil (matched: smiles)

dsgdb9nsd_000014 C2H6O
9 atom · μ : 1.41 · gap: 0.342

← Sebelumnya 1 / 1 Berikutnya →

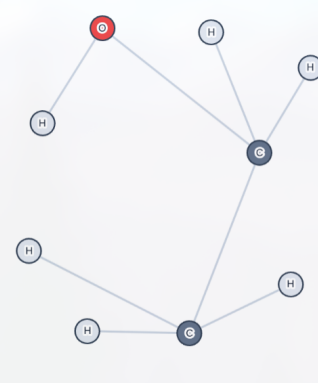
Render dari Notasi

Ethanol

dsgdb9nsd_000014 · C2H6O · 9 atom · 8 ikatan · SMILES: CCO

+ tambah ke pembandingan

GRAF 2D Scroll untuk zoom · Drag untuk geser



PROPERTI

Rot. Const A (GHz)	35.0954	Rot. Const B (GHz)	9.3686
Rot. Const C (GHz)	8.1497	Dipole Moment (Debye)	1.4131
Polarizability (Bohr ³)	27.8700	HOMO (Hartree)	-0.2619
LUMO (Hartree)	0.0798	HOMO-LUMO Gap (Hartree)	0.3417

Gambar 4.5: Pencarian Tier 1b: input SMILES CCO mengembalikan satu molekul (etanol) lewat hash map `smiles_idx`.

QM9 Graph Explorer Bandingkan

DAFTAR MOLEKUL

dsgdb9nsd_000001

dsgdb9nsd_000001 CH4
5 atom · μ : 0.00 · gap: 0.505

dsgdb9nsd_000002 H3N
4 atom · μ : 1.63 · gap: 0.340

dsgdb9nsd_000003 H2O
3 atom · μ : 1.85 · gap: 0.361

dsgdb9nsd_000004 C2H2
4 atom · μ : 0.00 · gap: 0.335

dsgdb9nsd_000005 CHN
3 atom · μ : 2.89 · gap: 0.380

dsgdb9nsd_000006 CH2O
4 atom · μ : 2.11 · gap: 0.226

dsgdb9nsd_000007 C2H6
8 atom · μ : 0.00 · gap: 0.443

dsgdb9nsd_000008 CH4O
6 atom · μ : 1.53 · gap: 0.344

← Sebelumnya 1 / 6695 Berikutnya →

Render dari Notasi

Methane

dsgdb9nsd_000001 · CH4 · 5 atom · 4 ikatan · SMILES: C

+ tambah ke pembandingan

GRAF 2D Scroll untuk zoom · Drag untuk geser



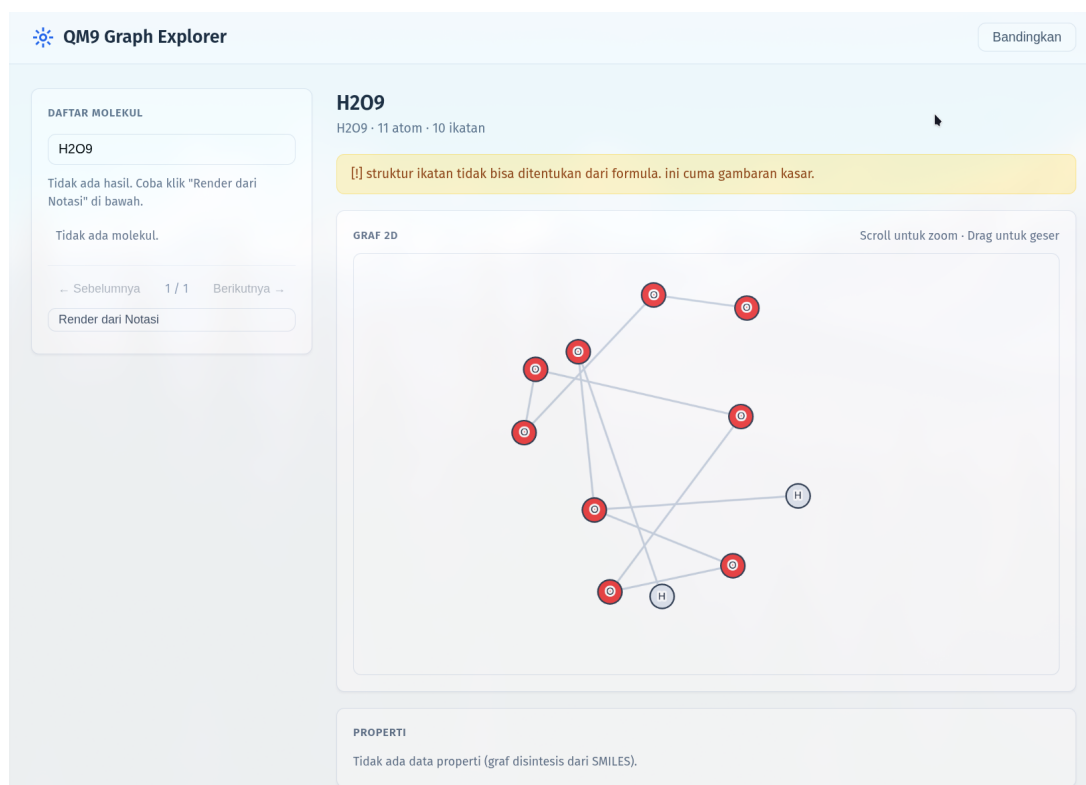
PROPERTI

Rot. Const A (GHz)	157.7118	Rot. Const B (GHz)	157.7100
Rot. Const C (GHz)	157.7070	Dipole Moment (Debye)	0.0000

Gambar 4.6: Pencarian Tier 2 (*fallback*): substring pada `mol_id` dipakai jika dua tier sebelumnya tidak cocok.

4.4 Fallback Render dari Notasi

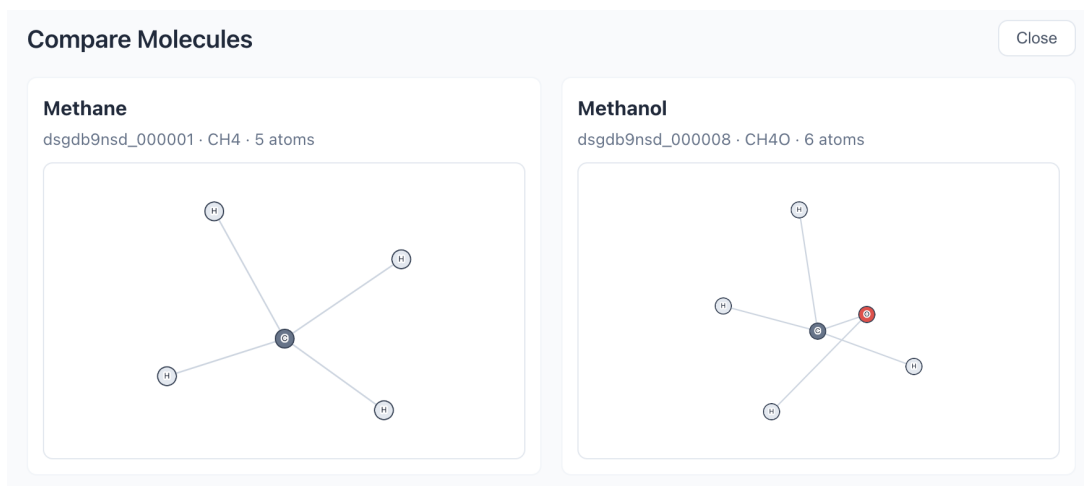
Untuk molekul yang tidak tersedia di dataset, baik karena formulanya tidak alami (mis. H_2O_9) maupun SMILES sembarang yang ingin diuji oleh pengguna, sistem menyediakan tombol “Render dari Notasi”. Hasilnya berupa graf sintetis yang dibangun dari parser SMILES (memakai stack untuk *branch*) atau parser formula naif. Karena objek yang dihasilkan tidak punya koordinat 3D dan properti kuantum, panel propertinya kosong dan ditandai dengan flag `synthetic = True` disertai pesan peringatan.



Gambar 4.7: Mode *fallback*: graf sintetis yang dirender dari notasi yang tidak ada di dataset.

4.5 Mode Bandingkan

Pengguna diberi keleluasaan untuk memilih sampai empat molekul sekaligus, lalu menampilkan grafnya secara berdampingan dalam satu layar.



Gambar 4.8: Mode bandingkan: 4 graf molekul ditampilkan berdampingan.

4.6 Performa yang Diharapkan

Mengacu pada implementasi yang sudah berjalan, target performa sistem ditetapkan sebagai berikut:

- *Startup* aplikasi: **kurang dari 1 detik** untuk memuat `index.json` berukuran 63 MB.
- Lookup berdasarkan ID, formula, atau SMILES: $O(1)$, hasil keluar dalam orde milidetik.
- Pembukaan detail molekul untuk pertama kalinya: **40 sampai 60 ms**, mencakup proses parse `.xyz`, inferensi ikatan, dan perhitungan tata letak 2D.
- Pembukaan detail molekul yang sama untuk kali kedua: **di bawah 10 ms**, sebab data sudah tersedia di cache.
- *Render fallback* dari SMILES: **30 sampai 100 ms**, bergantung ukuran molekul yang di-input.

Name	Status	Type	Initiator	Size	Time
dsgdb9nsd_000003	200	fetch	app.js:26	1.0 kB	4 ms
dsgdb9nsd_000009	200	fetch	app.js:26	1.7 kB	52 ms
dsgdb9nsd_000011	200	fetch	app.js:26	1.7 kB	9 ms
dsgdb9nsd_000013	200	fetch	app.js:26	2.3 kB	53 ms
dsgdb9nsd_000014	200	fetch	app.js:26	2.0 kB	3 ms
render-formula	200	fetch	app.js:26	1.7 kB	44 ms

Gambar 4.9: Bukti performa di tab Network DevTools. Hit pertama (cache miss) untuk molekul yang baru dibuka berkisar 40–60 ms, sedangkan hit ulang (cache hit) di bawah 10 ms.

BAB 5

ALGORITMA / URUTAN Pengerjaan

Bab ini memaparkan algoritma utama yang dipakai sistem dalam format **input, proses, output**. Urutan algoritma disusun mengikuti alur eksekusi yang sebenarnya, dimulai dari tahap setup awal lalu berlanjut ke operasi runtime.

5.1 Algoritma 1: Membangun Indeks (`index_builder.py`)

Input: Direktori `data/qm9_raw/` yang memuat 133.885 berkas `.xyz`.

Proses:

1. Siapkan tiga *hash map* dalam keadaan kosong, masing-masing diberi nama `meta`, `formula_idx`, dan `smiles_idx`.
2. Iterasi dilakukan ke seluruh berkas `.xyz`, dengan tahapan sebagai berikut:
 - (a) Bagian *header* dibaca secara ringan, mencakup jumlah atom, 16 properti kuantum, daftar elemen, dan string SMILES.
 - (b) Rumus kimia dihitung memakai notasi Hill, yaitu unsur C ditempatkan paling depan, dilanjutkan H, lalu sisanya disusun secara alfabetis.
 - (c) Metadata yang sudah diperoleh disimpan ke `meta[mol_id]`.
 - (d) Identitas molekul ditambahkan ke daftar `formula_idx[formula]`.
 - (e) Pasangan SMILES dengan `mol_id` dicatat melalui `smiles_idx[smiles] = mol_id`.
3. Ketiga *hash map* di-*serialize* ke berkas `data/qm9_processed/index.json`.

Output: Berkas `index.json` dengan ukuran sekitar 63 MB, siap dipakai oleh aplikasi.

Kompleksitas: Waktu prosesnya sebesar $O(N \cdot k)$, dengan N menyatakan jumlah file dan k rata-rata ukuran header (konstan kecil). Pengukuran empiris menunjukkan tahap pembangunan indeks selesai dalam waktu kurang lebih 3 detik untuk 133.885 file. Adapun total waktu `setup.py` secara keseluruhan, mencakup unduh dataset, ekstrak 134 ribu `.xyz`, dan pembuatan indeks, berkisar 41 detik.

5.2 Algoritma 2: Pencarian Molekul (`search`)

Input: *Query string* `q` yang dimasukkan pengguna, sebagai contoh `H2O`, `CCO`, atau `dsgd b9nsd_000003`.

Proses:

1. Normalisasi pada q , meliputi pengubahan *subscript* Unicode menjadi digit biasa serta penyamaan kapitalisasi tiap elemen kimia.
2. **Tier 1:** nilai q dicek di `formula_idx`. Bila cocok, sistem mengembalikan daftar `mol_id` yang berkaitan.
3. **Tier 1b:** q kembali dicek di `smiles_idx`. Jika ditemukan, satu `mol_id` dikembalikan.
4. **Tier 2 (fallback):** pencarian beralih ke pencocokan *substring* pada `mol_id` dan `name`, sesuai kata yang diketik pengguna.

Output: Daftar metadata molekul yang disertai indikator `matched_by` dengan tiga kemungkinan nilai, yaitu `formula`, `smiles`, atau `substring`.

Kompleksitas: Tier 1 dan 1b berjalan dalam $O(1)$ karena memanfaatkan *hash map lookup*. Tier 2 perlu waktu $O(n)$ dengan n adalah jumlah molekul, namun jalur ini hanya dieksekusi kalau Tier 1 dan 1b sama-sama *miss*.

5.3 Algoritma 3: Buka Detail Molekul (*Lazy Load*)

Input: `mol_id` yang dipilih oleh pengguna.

Proses:

1. Periksa keberadaan `mol_id` di cache pickle (`self._cache`). Bila sudah ada, hasilnya langsung dikembalikan.
2. Validasi `mol_id` di `meta`. Apabila tidak ditemukan, sistem mengembalikan `None`.
3. Berkas `data/qm9_raw/{mol_id}.xyz` dibaca.
4. Algoritma **infer_bonds** (Algoritma 4) dijalankan untuk menyimpulkan daftar ikatan dari koordinat 3D.
5. Algoritma **compute_2d_layout** (Algoritma 5) dipanggil agar posisi tiap atom pada bidang 2D dapat ditentukan.
6. Objek molekul lengkap dibentuk dengan komponen *nodes*, *edges*, dan *properties*.
7. Hasil pembentukan disimpan ke cache lewat `self._cache[mol_id] = molekul`.
8. Cache yang sudah diperbarui di-*flush* ke berkas `cache.pkl`.

Output: Objek molekul utuh yang berisi *nodes*, *edges*, koordinat 2D, dan ke-16 properti kuantum.

Kompleksitas: Bila cache mengalami *hit*, prosesnya cuma $O(1)$. Bila *miss*, total kompleksitasnya menjadi $O(n^2) + O(n^3)$ yang berasal dari inferensi ikatan dan algoritma layout. Variabel n di sini menyatakan jumlah atom. Pada QM9, nilainya tidak lebih dari 29, sehingga praktis bisa dianggap konstan.

5.4 Algoritma 4: Inferensi Ikatan (`infer_bonds`)

Input: Daftar elemen `atoms` sebagai contoh `["C", "H", "H", "O"]`, beserta matriks koordinat 3D `coords` berdimensi $n \times 3$.

Proses:

1. Variabel `bonds` disiapkan dalam keadaan kosong.
2. Iterasi dilakukan ke seluruh pasangan atom (i, j) dengan ketentuan $i < j$. Pendekatan segitiga atas ini dipakai karena grafnya tidak berarah. Untuk setiap pasangan, langkahnya:
 - (a) Nilai jari-jari kovalen `r1` dan `r2` diambil dari *hash map* `COVALENT_RADII` dengan kompleksitas $O(1)$.
 - (b) Jarak Euclidean `dist` antara `coords[i]` dan `coords[j]` dihitung.
 - (c) Bila `dist` bernilai lebih kecil daripada `r1 + r2 + tolerance`, dengan `tolerance = 0,45 Å`, maka tuple $(i, j, dist)$ ditambahkan ke daftar `bonds`.

Output: Daftar *tuple* (i, j, jarak) yang menggambarkan ikatan kimia antar atom.

Kompleksitas: Berada di kelas $O(n^2)$ sebab seluruh pasangan atom diperiksa. Pada dataset QM9 dengan $n \leq 29$, eksekusinya cepat dan biasanya selesai di bawah 1 ms.

5.5 Algoritma 5: Tata Letak 2D (`compute_2d_layout`)

Input: Daftar atom beserta daftar ikatan yang dihasilkan oleh `infer_bonds`.

Proses:

1. Objek `nx.Graph` berbasis *adjacency list* dibangun dengan menambahkan seluruh *node* dan *edge* ke dalamnya.
2. Algoritma **Kamada-Kawai** dicoba terlebih dahulu. Cara kerjanya adalah meminimalkan energi sistem pegas, di mana panjang ideal antar *node* dibuat sebanding dengan *shortest path*.
3. Jika algoritma sebelumnya gagal, misalnya pada kasus graf yang tidak terhubung, sistem beralih ke **Fruchterman-Reingold** sebagai *fallback*.
4. Posisi yang sudah didapat dinormalisasi. Caranya, rentang x dan y dihitung, lalu skala disesuaikan ke kisaran ± 150 piksel, dan terakhir titik tengahnya dipusatkan ke origin.

Output: *Hash map* yang memetakan `node_id` ke pasangan koordinat (x, y) dalam satuan piksel.

Kompleksitas: Algoritma Kamada-Kawai memerlukan $O(n^3)$ untuk komputasi *shortest path* dengan Floyd-Warshall, kemudian $O(n^2)$ per iterasi Newton-Raphson. Adapun Fruchterman-Reingold beroperasi pada $O(\text{iter} \times n^2)$. Khusus QM9 yang memiliki $n \leq 29$, kedua algoritma ini selesai dalam waktu di bawah 50 ms.

5.6 Algoritma 6: Render *Fallback* dari Notasi

Input: *String* berupa formula (sebagai contoh H2O) maupun SMILES (sebagai contoh CCO) yang dimasukkan pengguna, dipakai khusus untuk molekul yang *tidak tersedia* di dataset.

Proses (jalur formula):

1. Formula di-*parse* dengan *regex* untuk mengekstrak pasangan elemen beserta jumlahnya.
2. Atom dipisahkan ke dalam dua kategori, yaitu atom berat (non-H) dan atom hidrogen.
3. Ikatan dibangun secara naif. Antar atom berat dirangkai secara linier, kemudian atom hidrogen didistribusikan ke atom berat dengan pola *round-robin*.
4. Tata letak 2D dihitung lewat **compute_2d_layout** (Algoritma 5).

Proses (jalur SMILES):

1. String SMILES di-*tokenize* memakai *regex* yang mencakup atom organik, *bracket*, *bond*, *branch*, dan *ring closure*.
2. Daftar atom dan ikatan disusun dengan bantuan *stack* untuk menangani *branch*, ditambah *hash map* sebagai pencatat *ring closure*.
3. Atom hidrogen secara *implicit* ditambahkan dengan menyesuaikan valensi tiap atom.
4. Tata letak 2D dihitung kembali memakai algoritma yang sama seperti pada jalur formula.

Output: Objek molekul yang sifatnya sintetis. Objek ini ditandai dengan *flag* `synthetic = True` dan disertai pesan peringatan. Tidak ada koordinat 3D maupun properti kuantum yang menyertai.

Kompleksitas: Tahap *parsing* memerlukan $O(n)$, sedangkan tahap layout memerlukan $O(n^3)$. Untuk molekul yang tergolong kecil dengan jumlah atom kurang dari 30, total waktu eksekusinya berkisar 30 sampai 100 ms.

BAB 6

LAMPIRAN

6.1 Repositori Kode

Kode sumber proyek ini sepenuhnya dipublikasikan di GitHub pada alamat berikut:

```
https://github.com/alfathir27/sda-project
```

Isi repositori meliputi kode *backend* berbasis Python dengan FastAPI, kode *frontend* yang dibangun dari HTML, JavaScript, dan Cytoscape.js, lalu skrip `setup.py` sebagai otomatisasi instalasi (membuat *virtual environment*, mengunduh dataset, dan membangun indeks). Dokumentasi pendukung berupa `README.md` serta laporan $\text{\LaTeX}$ juga turut disertakan.

6.2 Cara Menjalankan

Begitu repositori sudah di-*clone*, semua tahap penyiapan dapat dijalankan lewat satu perintah saja. Perintah ini mencakup pembuatan *virtual environment*, instalasi dependensi, pengunduhan dataset berukuran 83 MB, ekstraksi 134 ribu berkas `.xyz`, sampai pembangunan indeks:

```
python setup.py
```

Setelah seluruh persiapan rampung, jalankan *server* berikut:

```
backend/.venv/bin/uvicorn main:app \
    --app-dir backend --port 8000
```

Aplikasi siap diakses melalui `http://localhost:8000` pada peramban.

Bibliografi

- [1] Lu, Y., Chen, M., Zhang, Q., Zhang, J., Zhang, C., Xu, S., & Bi, Q. (2026). Adaptive edge-aware graph convolutional with multi-task learning for simultaneous prediction of material properties. *npj Computational Materials*, 12(49). <https://doi.org/10.1038/s41524-025-01917-7>
- [2] Ramakrishnan, R., Dral, P. O., Rupp, M., & von Lilienfeld, O. A. (2014). Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1(140022). <https://doi.org/10.1038/sdata.2014.22>
- [3] Kamada, T., & Kawai, S. (1989). An algorithm for drawing general undirected graphs. *Information Processing Letters*, 31(1), 7–15. [https://doi.org/10.1016/0020-0190\(89\)90102-6](https://doi.org/10.1016/0020-0190(89)90102-6)
- [4] Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference (SciPy)*, 11–15. <https://doi.org/10.25080/TCWV9851>